

EXP : 19

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix

# Sample bank loan dataset
data = {
    'Income': [25000, 30000, 45000, 50000, 60000, 75000,
               80000, 90000, 28000, 40000, 55000, 70000],

    'LoanAmount': [200000, 150000, 180000, 250000, 200000, 300000,
                   250000, 350000, 220000, 180000, 200000, 280000],

    'CreditScore': [550, 580, 620, 650, 680, 720,
                    750, 780, 560, 630, 690, 730],

    'Age': [25, 30, 35, 40, 45, 50,
            55, 60, 28, 36, 42, 48],

    'LoanApproved': [0, 0, 1, 1, 1, 1,
                     1, 1, 0, 1, 1, 1]
}

# Create DataFrame
df = pd.DataFrame(data)

# Input features
X = df[['Income', 'LoanAmount', 'CreditScore', 'Age']]
```

```
# Target variable
y = df['LoanApproved']

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

# Create Naive Bayes classifier
model = GaussianNB()

# Train the model
model.fit(X_train, y_train)

# Predict test data
y_pred = model.predict(X_test)

# Display actual and predicted values
print("Actual Values:")
print(y_test.values)

print("\nPredicted Values:")
print(y_pred)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:", accuracy * 100, "%")

# Display confusion matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
# Predict a new customer's loan eligibility

new_customer = [[60000, 200000, 700, 45]]

prediction = model.predict(new_customer)

if prediction[0] == 1:
    print("\nNew Customer: Loan Approved")
else:
    print("\nNew Customer: Loan Not Approved")
```

---

```
... Actual Values:
[1 1 0]

Predicted Values:
[1 1 0]

Accuracy: 100.0 %

Confusion Matrix:
[[1 0]
 [0 2]]

New Customer: Loan Approved
```